GAME DESIGN DOCUMENT

University of Macedonia
Department of Applied Informatics
M.Sc. in Software Development & Cloud

Serious Game Design & Development, Summer Semester

Archontis E. Kostis - *mai26037@uom.edu.gr*
June 2026

Built with Greenfoot
Source Code: github.com/ArchontisKostis/codebot-maze-serious-game

Permission to share project with other students	**YES**

Table of Contents

1. Introduction

“RIVETS: Robot Programmer” is a serious game whose purpose is education, specifically, teaching the foundations of imperative programming and computational thinking to learners with little to no prior coding experience. The game teaches the basics of programming through play, as the player writes short programs to guide a robot through 15 maze chambers, using a custom programming language.

Figure 1. Game Logo

1.1 Game at a glance

Attribute	Value
Title	RIVETS: Robot Programmer
Purpose	Teach the foundations of programming and computational thinking
Genre	2D top-down, tile-based puzzle / coding game
Content	A custom-made imperative language (four move commands + counted loops), 15 built-in chambers, plus a free-play / custom-level mode
Platform	Desktop (Windows, macOS, Linux)
Technologies	Java & Greenfoot

2. Reference Points & Backround

Most games that teach programming fall into two categories: *block-based*, where pre-made blocks are snapped together like puzzle pieces, and *text-based*, where the learner writes real code [1]. Project Rivets falls in the second category, the table bellow compares our game with the best-known ones, including those used in Greek schools.

Tool	What it is	Relation to Project Rivets
Block-based tools		
Scratch (MIT)	A block-based environment for roughly ages 10-18: programs, stories and games are built by snapping puzzle-piece blocks together.	Keeps Scratch's low-friction visual spirit but replaces blocks with typed text.
Blockly, Code.org Maze, Lightbot	Block- or icon-based maze puzzles: assemble a short sequence to move a character to a goal.	As with Scratch, the blocks are replaced with a real typed language, run by a lexer-parser-interpreter pipeline.

Tool	What it is	Relation to Project Rivets
Text-based tools		
Karel the Robot	A robot on a grid driven by a small imperative language.	The closest relative. Project Rivets embeds the editor, terminal and robot in one game and adds a story plus a star/abstraction score that also grades code quality and not just completion.
MicroWorlds Pro	A Logo/turtle microworld used in schools (including Greece) for teachiing programming.	Shares the idea of controlling an agent in a microworld, but is puzzle and goal driven with an explicit objective rather than open turtle drawing.

3. Audience, Goals and Expected Benefits

3.1 Target audience

Dimension	Description
Age	8 and over. Aimed mainly at school students. Also suitable for anyone older who is new to programming.
Prior knowledge	None assumed. Every concept and keyword is introduced from zero. There is no programming toolchain to install and no syntax to know in advance.
Gender / culture	Gender-neutral. The robotics lab setting and abstract puzzles carry no gendered or culturally specific assumptions. All in-game text is in clear, simple English.
Context of use	Self-directed play at home, or guided use in a classroom or lab as a first taste of programming. Fully offline.
Also useful for	Teachers who want a low-friction way to show sequencing, loops and debugging and who can build their own mazes with the level editor.

3.2 Learning objectives and content

The overall goal of the game is to take a complete beginner and, through the game, lead them to write and reason about real programs. The content is delivered in four groups of increasing abstraction (sequencing, then counted iteration, then nested iteration).

Level group	What the player learns
Group A: Command Discovery (Chambers 1-4)	That a program is an explicit, ordered sequence of instructions executed literally + the four movement commands.
Group B: Sequencing Mastery (Chambers 5-7)	Composing longer multi-direction routes and planning ahead (decomposition and algorithm design)
Group C: Loop Introduction (Chambers 8-10)	Spotting repetition and expressing it with `repeat(N) … end` (loop is a cleaner alternative of a repeated sequence)
Group D: Structural Reasoning (Chambers 11-15)	Seeing two-dimensional regularity, using nested loops, and judging when a loop fits and when sequential code is still needed

3.3 Expected benefits and learning outcomes

After finishing the campaign, a player should be able to:

1. read a problem (a maze) and **break it down** into an ordered sequence of explicit steps.

2. write, run and **debug** a real program, reading plain-language error messages and fixing them.
3. spot **repetition** and express it with a counted loop instead of repeating commands by hand.
4. use **nested loops** to capture two-dimensional, structured patterns.

4. Design Approach

4.1 Framework: CMX

The game's design is based on CMX [2], a framework created specifically for educational programming games.

CMX concern	How Project Rivets addresses it
Learning objectives	Teach programming and computational thinking, with concrete per-group goals.
Pedagogy / strategy	Problem-based: every chamber is solved by building a program.
Learning content	Sequencing, then counted iteration, then nested iteration, across 15 chambers.
Scenario & characters	A researcher trains the robot RIVETS, with narration.
Virtual world	A 960×720 simulation-lab interface with a 24×20 tile maze.
Programming editor	An in-game code editor with syntax highlighting and a live terminal.
Language & compiler	A real Robot Script Language run by full a lexer, parser and interpreter pipeline.
Scaffolding	Command unlocks, onboarding overlays, intro pop-ups, and plain-English errors.
Awards & incentives	Coins, a 0-3 star rating per level (chamber) and a final classification.
Reflection & evaluation	A star overview, the terminal trace for players, and an evaluation plan for the serious game.

4.2 Learning strategy and scaffolding

The approach is problem-based: each chamber is a problem whose only solution is a working program, so the player learns by building rather than being told. The core idea is that the robot does *exactly* as is told (what the program says), and not what the player “meant”. A lesson players absorb the first time the robot walks straight into a wall. Support for beginners is built into the game progression:

- **Commands unlock one at a time:** `moveRight` first, then `moveDown`, and `repeat` at last. This is so the language never arrives all at once to the player.
- **An onboarding overlay** walks new players through the editor, the RUN/RESET buttons and the terminal.
- **Each chamber opens with a short intro pop-up:** the story, the available commands with one example, and an objective.
- **Errors are written in plain English**, naming the line and the problem (e.g. **“**`Line 3: Expected 'end' to close repeat(8)`**”**), so mistakes are a lesson rather than a wall.

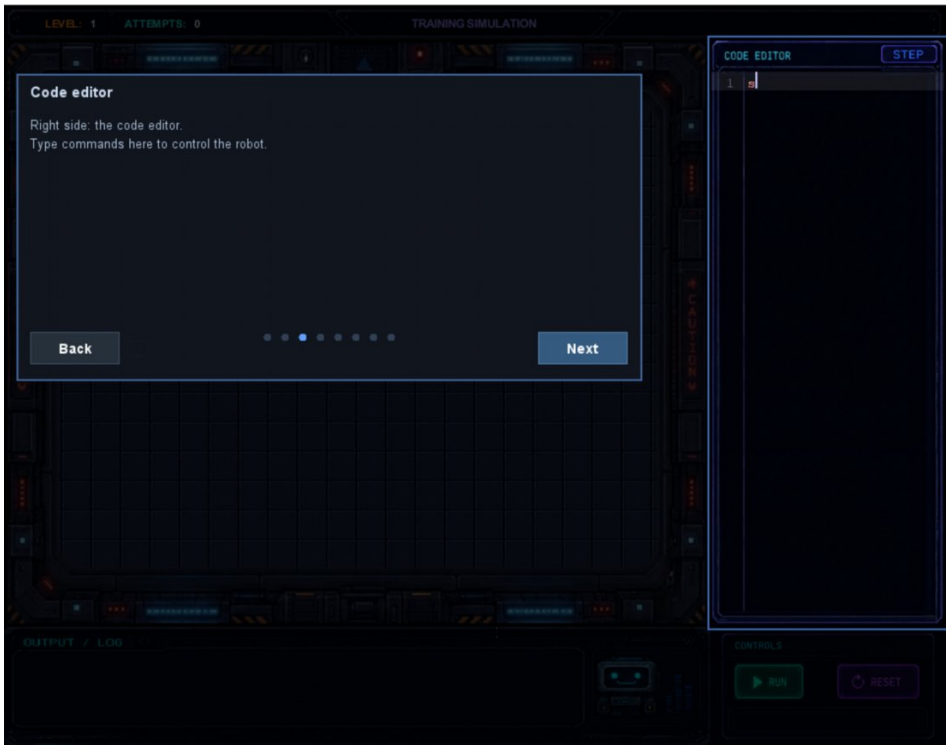

Figure 2. First-run onboarding spotlights each region of the screen in turn.

4.3 Key design decisions

Decision	Why
Typed text, not drag-and-drop blocks	Players write and read real code, so the skill transfers to actual programming.
A tiny six-keyword language	Mastery is reachable and cognitive load stays very low. Players can memorize the whole language in under a minute.
A real lexer-parser-interpreter (not string matching)	Players run genuine programs, and errors are real and explained in plain English. This is not directly relative to the gameplay experience or pedagogy but it makes the enhancement of the language easier and the overall implementation cleaner and more maintainable.
One instruction animated per frame	Execution and iteration are visible, building an accurate mental model.
Commands unlock one at a time	The language is introduced gradually, never all at once.
Loops are rewarded but never required	Every chamber is finishable without the use of loops. Usage of loops is rewarded as a clean code approach.
A separate web level editor	Teachers and players can create and share new chambers.

5. Narrative and Game World

5.1 Premise

The player is a new researcher at the **Axiom Robotics Training Division**. Their job is to train **RIVETS**, the lab's most advanced robot, by writing the programs it runs through fifteen sealed simulation chambers. In the story, every program is a training input, RIVETS absorbs the structure and quality of each session, and the accumulated quality decides what kind of machine it will become.

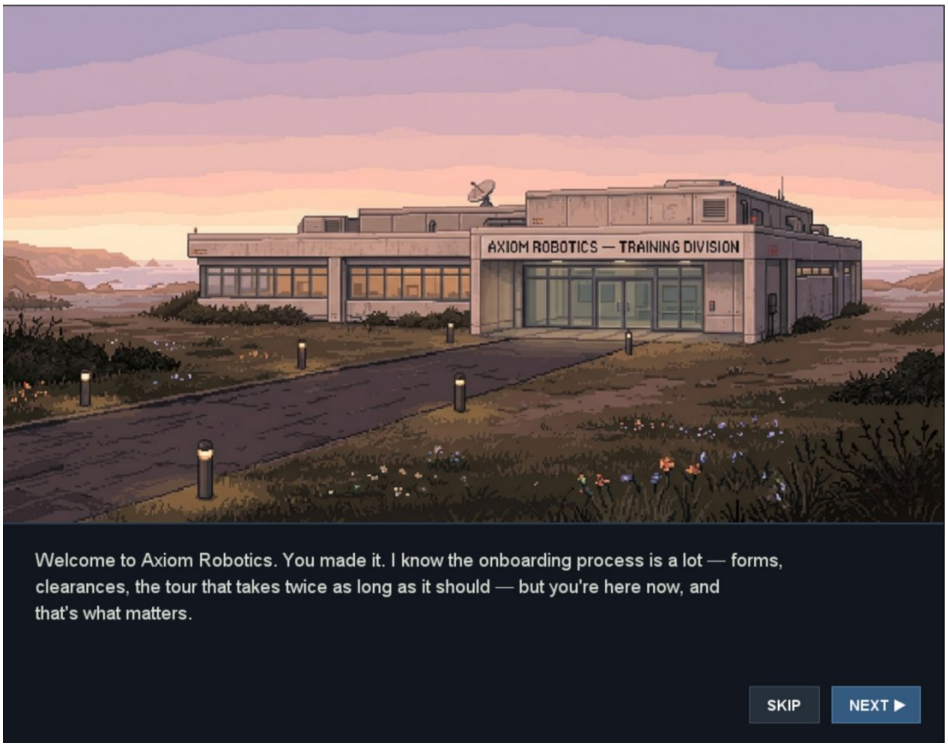

Figure 3. Intro Slide 1

5.2 Characters

Character	Role
RIVETS (the robot)	The prototype being trained and the player's on-screen agent. It runs the player's program exactly as written.
The researcher (player)	An unnamed stand-in for whoever is at the keyboard. There is no avatar, the player's presence is the code in the editor.
Helen	A senior researcher who narrates the game.

5.3 Story arc

1. Arrival	An 8 slide intro, narrated by Helen, sets up the lab, the project and the stakes.
2. Training	Across the 15 chambers, short pop-ups frame RIVETS's progress and unlock each new command.
3. Classification	When every chamber is complete, the lab issues a final verdict on RIVETS intelligence based on the player's total score

6. Gameplay, Rules and the Robot Script Language

6.1 The core loop

The game runs on a tight loop the player repeats over and over:

1. **Read** the maze: find the robot's start, the goal, any coins, and the walls.
2. **Write** a program in the editor using the commands unlocked so far.
3. **Run**: the code is lexed, parsed and then executed one instruction per frame, so the robot steps tile-by-tile while the terminal logs each action.
4. **Observe**: watch where the robot succeeds, gets blocked, or misses a coin. Read any errors in the terminal.

5. **Refine**: RESET returns the robot to its start (code is not affected), the player edits and re-runs the program.

6.2 The Robot Script Language (RSL)

The Robot Script Language is a tiny custom made imperative language we created with six . The language is case and whitespace insensitive, so players can format freely and are never tripped up by capitalisation.

Source text	Meaning	Token
`moveUp`, `moveDown`, `moveLeft`, `moveRight`	Move the robot exactly one tile in that direction.	`MOVE_UP` / `DOWN` / `LEFT` / `RIGHT`
`repeat(N) ... end`	Run the enclosed block `N` times (`N` must be 1-100). May be nested.	`REPEAT ... END`
`1–100`	The loop count.	`NUMBER`

Robot Script - grammar (EBNF)

```
program    ::=  statement* EOF
statement  ::=  move | repeat
move       ::=  "moveUp" | "moveDown" | "moveLeft" | "moveRight"
repeat     ::=  "repeat" "(" NUMBER ")" statement* "end"
```

6.3 Controls

Input	Action
Typing, Enter, Backspace, Tab	Edit the program in the code editor (with live syntax highlighting).
RUN button	Lex and parse the editor text and start the robot's animated run.
RESET button	Return the robot to its start tile and clear the terminal (the code is kept)
Menu buttons	START, FREE PLAY, SETTINGS, INSTRUCTIONS, CREDITS on the home screen.

6.4 Objectives, win and fail

Win	The robot reaches the goal, a sound plays, a completion overlay shows the star rating, and the next chamber unlocks.
Classification	When every chamber is complete, the lab issues a final verdict on RIVETS based on the player's total score
No hard fail	The program simply may not reach the goal. The player presses RESET and tries again.

6.5 Rules summary

Rule	Detail
Grid	24×20 tiles. The robot occupies one tile and moves one tile per instruction.
Walls	Impassable. A move into a wall is cancelled and reported.
Coins	Optional collectibles on floor tiles. Collected by stepping on them, they also feed the score.
Loops	Repeat count must be 1-100 and blocks may be nested to any depth. The `end` token closes the most recent repeat.

Rule	Detail
Goal	Reaching the goal completes the chamber. As mentioned, every chamber can be finished without loops but loops are rewarded.

7. Progression, Scoring and Rewards

7.1 Scoring model

Every completed chamber yields a score from 0 to 100, produced by one of two scorers (`CompletionScorer` & `AbstractionScorer`), and that score becomes a 0-3 star rating.

Chambers 1 to 7 are scored on based **completion**: how many coins were collected and how few attempts it took. While, Chambers 8 to 15 are scored on **abstraction**: whether the player used loops, and how deeply nested. A player who brute-forces a loop chamber still finishes, but the low rating is an invitation to come back and improve.

Sequential solution (1 star)

```
// reach a goal 5 right, 2 down
moveRight
moveRight
moveRight
moveRight
moveRight
moveDown
moveDown
```

Loop solution (3 stars)

```
// same path, structured
repeat(5)
  moveRight
end

repeat(2)
  moveDown
end
```

The same path, solved two ways (a sequence and a loop)

7.3 Final classification

The collected Stars add up across all fifteen chambers (maximum 45). At the last chamber (Chamber 15) the game issues the final robot's classification. This is the story's climax, and the game's headline achievement.

Classification	Stars	Meaning
Type III: Expert Programmer	40-45	Consistently optimal, well-structured solutions.
Type II: Advanced Programmer	28-39	Mostly correct with solid use of loops.
Type I: Apprentice Programmer	15-27	The fundamentals proved but little to no abstraction was used.
Training Incomplete	0-14	Not enough chambers cleared to classify.

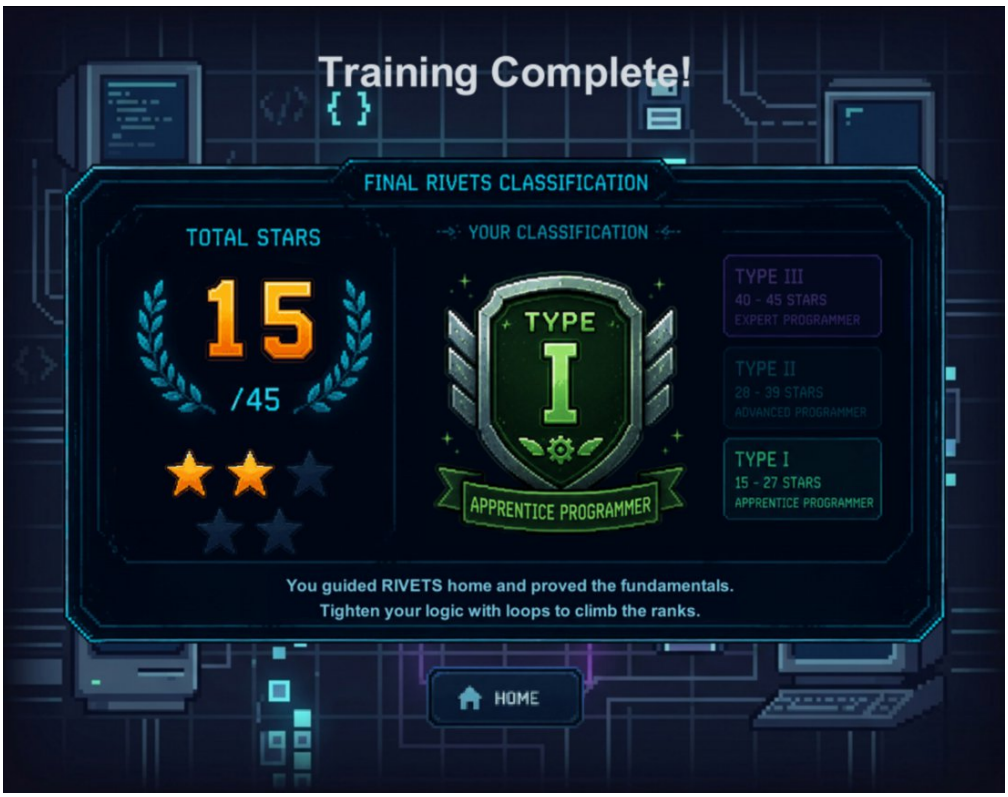

Figure 5. The final classification screen (here: Type I).

7.4 Progress and free play

Progress stays visible throughout the game. The HUD shows the current chamber and attempt count while on the end of every level an overview shows the stars earned. Beyond the campaign, **Free Play** allows users to load custom mazes (which don't affect campaign progress) made with the accompanying level editor (see Section 10).

8. User Interface and Audio

8.1 Screen layout

The game window is 960×720 px, split into four always visible regions, so the player never loses sight of the maze while coding, keeping code and effect side by side.

Region	Size (px)	Purpose
Game area	720 × 600	The 24×20 tile maze with the robot, walls, coins and goal.
Code editor	240 × 600	Multi-line editor with live syntax highlighting and line numbers (right side).
Terminal	720 × 120	Execution log, errors and completion / star messages (bottom-left).
Controls	240 × 120	RUN & RESET buttons (bottom-right).

Figure 6. The gameplay screen: maze (left), code editor (right), terminal (bottom-left) and controls (bottom-right).

8.2 Audio

Event	Sound file	Source
Any button click	`generic-btn-click.mp3`	Mixkit
RUN pressed	`run-btn-click-sound.mp3`	Mixkit
RESET pressed	`reset-code-btn-click-sound.mp3`	Mixkit
Goal reached	`goal-reached-notification.mp3`	Mixkit
Lexer / parser error	`error-sound.mp3`	Mixkit

9. Technical Architecture and Implementation

9.1 Technologies

The game is written in Java using the Greenfoot framework, which supplies the World/Actor model, the game loop, image drawing and keyboard input. The game runs on Windows, macOS and Linux, is fully offline and the source code is version-controlled on GitHub.

9.2 Subsystems

The classes are grouped into loosely-coupled subsystems with clear responsibilities:

- **Worlds / screens:** Home, Intro, Loading, the main Simulation world, Load Custom and the Final Classification finale.
- **Language pipeline:** Lexer → Parser → Interpreter over a small AST, driven by a shared compilation/execution service.

- **Editor, input and terminal:** the in-game code editor, syntax highlighter, terminal manager and a paste field for custom levels.
- **Tile and level system:** a fixed 24×20 tile map with background and object layers, an ASCII map parser, and the custom-level parser/loader.
- **Scoring and progression:** the `LevelScorer` interface with the Completion and Abstraction implementations, plus the level and progress managers.
- **UI overlays:** pop-ups, onboarding, level complete and other overlays, plus the shared layout and theme.

To see all the used classes along with their roles and responsibilities in the game's codebase refer to Appendix A.

9.3 The language pipeline

The game runs the player's code through a real language pipeline, as shown in the figure below.

Figure 7. The Robot Script pipeline: text → tokens → AST → one move per frame.

- `Lexer`: scans the raw text into a flat list of tokens (move keywords, repeat/end, parentheses and numbers), reporting unknown words with their line number.
- `Parser`: a recursive-descent parser that turns the tokens into an Abstract Syntax Tree of three node types: `Program`, `MoveStatement` and `RepeatStatement`. It validates the loop count (1-100) and that every repeat has a matching end. An example syntax tree for reference is shown in Figure 8.
- `Interpreter`: instead of running the program straight to the end, it keeps a stack of loop frames and returns exactly one `MoveStatement` for each game frame so the robot steps visibly.

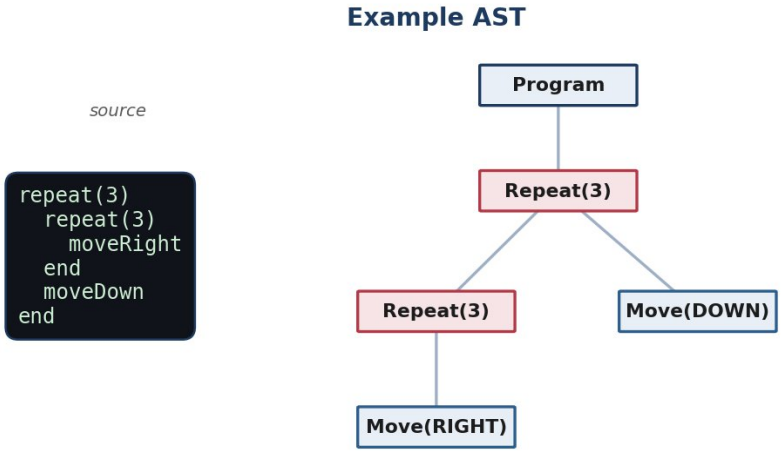

Figure 8. Example AST.

9.4 Developer and Instructor Cheats

Typing certain commands into the code editor triggers developer shortcuts (cheats). They let an instructor jump straight to any chamber to demonstrate a concept, or a developer test quickly, without playing through:

Command	Effect

Command	Effect
`goto N`	Jump straight to chamber `N`.
`skip`	Complete the current chamber.
`coins`	Collect every coin in the current chamber.
`stars`	Award 3 stars for the current chamber.
`end`	Jump to the final classification screen.
`resetAll`	Wipe all progress and return to chamber 1.

10. The Level Editor

Project Rivets ships with a custom level editor**.** The level editor is a separate web application for building and sharing custom chambers, hosted at codebot.archontis.gr/lvl-editor. It is a no-backend, static web app built in plain HTML5, CSS & JS. It reuses the game's art and palette, so editor and game look identical and exports levels as a `.lvl` text file or a single-line base64 share-code.

Users can paint a 24×20 grid with the palette (wall, floor, start, goal, coin) starting from either a blank shell or a ready-made template. The editor validates the level (exactly one start and one goal) before letting it be exported. The level can then be pasted into the game through Free Play → Load Custom.

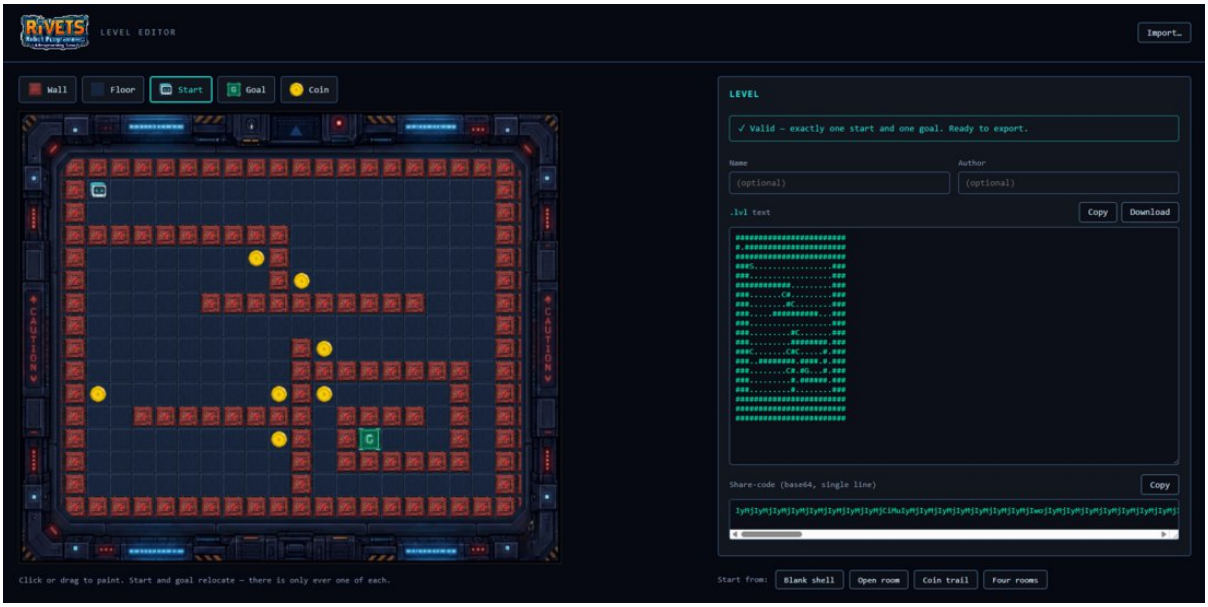

Figure 9. The companion web level editor

11. Evaluation Plan

To know whether the game meets its goals it would need to be tried with real players. A full study was outside the scope of this project (due to time and participant constraints), so no data was collected. The plan below details what a proper evaluation would involve, and is organized around six areas.

Area	Applied to Project Rivets
Player profile and interest	Prior programming exposure and gaming habits, captured before play, to put the results in context.
Performance	Smooth execution (no bugs or lag during runs).
Engagement and fun	Whether the story, art, animation and star system hold interest without overloading the player.

Area	Applied to Project Rivets
Interaction and difficulty	Whether challenge is matched to skill, and whether the feedback and language are clear and consistent.
Educational value	Whether players feel they learned the concepts the game tries to teach by playing (sequencing, loops and debugging), and would use such a game again.
Player results	The objective outcome: chambers completed, stars earned, and the final classification.

11.2 Sample questionnaire

A short questionnaire, answered on a 1-5 scale (1 = strongly disagree … 5 = strongly agree):

Question	1	2	3	4	5
The game held my attention and I wanted to keep playing.					
The robot's step-by-step movement helped me understand how my program runs.					
I understood when and why to use a repeat loop.					
The error messages helped me find and fix my mistakes.					
The difficulty increased at a comfortable pace.					
The star rating made me want to improve my solution.					
I would like to learn more programming with a game like this.					

Appendix A: Code Inventory

Worlds and screens

Class	Role	Source
`HomeWorld`	Main Menu World	Original
`IntroWorld`	Plays the intro opening slides before the campaign.	Original
`LoadingWorld`	Loading Screen shown between major transitions. Builds the next world lazily.	Original
`SimulationWorld`	The main gameplay world: hosts the maze, editor, terminal and controls.	Original
`FreePlayWorld`	Free-play menu: routes to Load Custom without touching campaign progress.	Original
`LoadCustomWorld`	Paste/import screen for custom levels	Original
`FinalClassificationWorld`	Campaign finale, composites the classification certificate.	Original

Language pipeline

Class	Role	Source
`Lexer`	Tokenises source text and reports lex errors with line numbers.	Original
`Parser`	Parser that builds the AST and validates repeat counts and matching end.	Original
`Interpreter`	Stack-based engine returning one move per frame for animated execution.	Original

Class	Role	Source
`Program`, `Statement`, `MoveStatement`, `RepeatStatement`	AST: root, marker interface, move leaf, and counted-loop node.	Original
`ProgramCompilationPipeline`	Shared program flow used by RUN and STEP.	Original
`ProgramExecutionService`	Drives stepwise execution and terminal logging during a run.	Original

Editor, terminal and input

Class	Role	Source
`CodeEditor`	In-game multi-line code editor (typing, caret, current-line highlight).	Original
`CodeSyntaxHighlighter`	Provides per-line syntax highlighting for the Robot Script Language.	Original
`TerminalManager`	Renders the output/terminal panel	Original
`LevelInputField`	Multi-line input field for the Load Custom screen	Original
`ClipboardAccess`	Reads OS clipboard text for the PASTE button (used for custom levels)	Original

Robot, actors and buttons

Class	Role	Source
`RobotActor`	The robot players program: steps one tile per command. Has collision.	Original
`Goal`, `Obstacle`, `Coin`	Goal tile, impassable wall, and collectible coin actors.	Original
`CommandButton`, `MenuButton`, `StartButton`	Clickable command buttons and menu buttons.	Original

Tile and level system

Class	Role	Source
`TileMap`, `TileCell`, `TileObjectKind`	Fixed 24×20 grid model: cells, background + one object per tile.	Original
`TileBackgroundId`, `TileBackgroundRegistry`	Background-tile identifiers mapped to artwork.	Original
`TileActorLayer`, `GameAreaTilePainter`	Spawns actors per tile and paints background tiles.	Original
`GameAreaConfig`, `GameScreenLayout`	Single source of truth for grid/pixel geometry and UI layout.	Original
`AsciiTileMapParse`, `TileLevelInstalle`, `ParsedTileLevel`	Build a tile-map from ASCII rows and install it into the world.	Original

Levels and level loading

Class	Role	Source
`Level` (interface)	Contract every chamber implements	Original

Class	Role	Source
`Level1` … `Level15`	The 15 built-in campaign chambers.	Original
`LevelManager`, `LevelProgressionController`	Track current level, unlocked commands and campaign progress.	Original
`LevelDefinition`, `LevelDocumentParser`, `LevelLoadResult`	Source-independent level data, custom level parser, level outcomes/errors.	Original
`CustomLevel`	A level built at runtime from pasted/loaded custom-level input.	Original

Scoring, overlays, story and state

Class	Role	Source
`LevelScorer`, `CompletionScorer`, `AbstractionScorer`, `ScorerKind`	0-100 scoring interface and its rules.	Original
`PopupOverlay`, `LevelCompleteOverlay`, `ProgramStopOverlayController`	Modal pop-ups, the level-complete star screen, and run-stop handling.	Original
`OnboardFlow`, `OnboardOverlay`, `OnboardStep`, `OnboardRegistry`	First-run spotlight onboarding tutorial.	Original
`IntroSlideData`, `IntroSlideRegistry`	Data for the narrated intro slides.	Original
`UiTheme`, `Settings`, `Sfx`	Shared palette, global settings, sound-effect helper.	Original
`CheatEngine`, `MyWorldSessionState`, `MyWorldCoinTracker`	Developer Cheat Engine, per-session state, coin tracking.	Original

Appendix B: Robot Script grammar (full)

```
(*  Robot Script - formal grammar (ISO/IEC 14977 EBNF)  *)

program     = { statement } , EOF ;
statement   = move | repeat-stmt ;
move        = "moveUp" | "moveDown" | "moveLeft" | "moveRight" ;
repeat-stmt = "repeat" , "(" , number , ")" , { statement } , "end" ;
number      = digit , { digit } ;   (* must be 1..100 *)

(*  keywords are case-insensitive; whitespace is ignored between tokens;
    one move runs per game frame; repeats may be nested.  *)
```